Politecnico di Torino

Master of Science (M.Sc.) in Computer Engineering

Academic Year 2025/2026

Graduation Session March 2026

Design and Implementation of a Cloud-Native Reactive Architecture

The Smart Travel Platform Case Study

Supervisor

Prof. Giovanni Malnati

Candidate

Federico Palazzi

Abstract

The rapid evolution of enterprise web applications has driven a paradigm shift from monolithic architectures to cloud-native *microservices* solutions. This transition is motivated by the need for greater scalability, resilience, and agility in the software development life cycle. This thesis presents the design, development and deployment of "*Smart Travel*", a comprehensive e-commerce platform dedicated to the sale of travel packages. The project serves as a proof of concept (PoC) developed within an industrial internship context, aiming to analyze the complexities and benefits of a modern and distributed software architecture.

The primary objective of this work is not merely the implementation of functional requirements (such as catalog management and order processing), but rather the critical evaluation of competing technologies and the automation of the release process. A central element of this research is the comparative analysis between two leading Java ecosystems: the established *Spring Boot* framework and the cloud-native *Quarkus* framework. Specifically, the project adopts a fully reactive approach for both of them, utilizing *Spring WebFlux* and *Quarkus Reactive* capabilities.

From an architectural perspective, the Smart Travel system is designed to ensure loose coupling and high availability. The backend adopts a microservices pattern supported by *RabbitMQ*, a message broker that facilitates asynchronous *event-driven* communication among services. Data persistence is managed through *MongoDB*, a NoSQL database chosen for its schema flexibility, which is particularly advantageous for handling diverse and evolving travel package structures compared to rigid relational models. On the client side, the user interface is delivered via a *Single Page Application (SPA)* developed with *Angular* and leveraging *RxJS* for reactive state management, ensuring a responsive and fluid user experience.

A significant portion of this thesis is dedicated to the "industrialization" of the software, bridging the gap between development and operations (*DevOps*). The project moves beyond local execution by implementing a robust *Continuous Integration and Continuous Delivery (CI/CD)* pipeline using *Jenkins*. This pipeline automates the entire delivery chain, including code checkout, compilation, container image creation and pushing to a private registry. The final deployment target is *OKD* (the community distribution of *Kubernetes* that powers *Red Hat OpenShift*). The thesis details the orchestration of containerized applications within the OKD cluster, managing critical aspects such as service discovery, route exposure and configuration management via `ConfigMaps` and `Secrets`.

In conclusion, the Smart Travel project demonstrates the viability and robustness of a heterogeneous, fully reactive microservices architecture. It successfully integrates different modern technologies into a cohesive ecosystem, proving that the

adoption of cloud-native principles, while introducing complexity in orchestration and consistency management, offers superior long-term benefits in terms of maintainability and scalability. The work provides a practical blueprint for modernizing legacy Java applications, offering concrete evidence of how the synergy between Reactive frameworks, Event-Driven patterns, and automated DevOps practices defines the state of the art in contemporary software engineering.

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Table of Contents

List of Figures VI

Glossary VIII

1 Introduction 1
- 1.1 Context 1
- 1.2 Goal 1
- 1.3 Thesis structure 1

2 Background and Technologies 3

3 System Architecture 4

4 Application Design and Implementation 5

5 Infrastructure Automation and Cloud Deployment 6

6 Conclusion 7

Bibliography 8

List of Figures

Glossary

CI/CD

Continuous Integration / Continuous Delivery

NoSQL

Not Only SQL

PoC

Proof of Concept

RxJS

Reactive Extensions for JavaScript

SPA

Single Page Application

Chapter 1

Introduction

1.1 Context

The industry shift from monolithic architectures to cloud-native microservices. Overview of the company, the Agile methodology used, and the goal of the "Smart Travel" project.

1.2 Goal

Developing a Proof of Concept (POC) e-commerce platform while critically evaluating modern Java frameworks (Spring Boot vs. Quarkus).

1.3 Thesis structure

The thesis is structured as follows:

- **Chapter 2: Background and Technologies** - Overview of microservices, reactive programming, and DevOps practices.
- **Chapter 3: System Architecture** - Architectural decisions, technology stack, and design patterns used in the Smart Travel project.
- **Chapter 4: Application Design and Implementation** - Detailed description of the development process, including code snippets and explanations of key components.
- **Chapter 5: Infrastructure Automation and Cloud Deployment** - Description of the CI/CD pipeline, Jenkins configuration, and deployment on OKD.

- **Chapter 6: Conclusion** - Summary of findings, limitations of the study, and future work directions.

Chapter 2

Background and Technologies

2.1 Microservices Architecture: Principles, benefits, and challenges.

2.2 Programming Paradigms: Imperative vs. Reactive programming models.

2.3 Event-Driven Architecture: Asynchronous communication and message brokers (focus on RabbitMQ).

2.4 The Java Ecosystem: 2.4.1 Spring Boot and Spring WebFlux

2.4.2 Quarkus and its cloud-native capabilities

2.5 Frontend Technologies: Single Page Applications (SPA), Angular, RxJS, and PrimeNG.

2.6 Database Technologies: NoSQL concepts and MongoDB.

2.7 API Paradigms: REST vs. GraphQL.

Chapter 3

System Architecture

3.1 Requirements Analysis: Functional and non-functional requirements of the Smart Travel platform.

3.2 High-Level Architecture: Decomposing the system into microservices (Users, Catalog, Orders, Notifications).

3.3 Data Modeling: Designing the MongoDB collections and handling flexible schemas.

3.4 Communication Patterns: * 3.4.1 Synchronous communication (REST/GraphQL)

3.4.2 Asynchronous communication (RabbitMQ events)

Chapter 4

Application Design and Implementation

4.1 Backend Development: * 4.1.1 Implementing the reactive services with Spring Boot

4.1.2 Implementing the reactive services with Quarkus

4.2 Frontend Development: Building the user interface, state management with RxJS, and connecting to the backend.

4.3 Core Workflows:

4.3.1 The catalog search and filtering mechanism

4.3.2 The payment integration flow

4.3.3 The event-driven notification system

Chapter 5

Infrastructure Automation and Cloud Deployment

5.1 The DevOps Culture: Bridging the gap between development and operations.

5.2 Continuous Integration & Continuous Delivery (CI/CD): Automating the build, test, and containerization process using Jenkins.

5.3 Container Orchestration with Kubernetes/OKD: * 5.3.1 OKD Cluster overview

5.3.2 Workload management: Pods and Deployments

5.3.3 Configuration and Security: ConfigMaps and Secrets

5.3.4 Networking: Services and Route exposure

Chapter 6

Conclusion

6.1 Technology Evaluation: Spring Boot vs. Quarkus: Comparing developer experience, startup times, resource consumption, and reactive capabilities.

6.2 Architectural Evaluation: Assessing the scalability, resilience, and maintainability of the implemented microservices and event-driven patterns.

6.3 Testing and Validation: Ensuring the functional requirements of the Smart Travel application were met.

Bibliography

[1] European Union Agency for Cybersecurity (ENISA). *ENISA Threat Landscape: Finance Sector*. 2025. URL: https://www.enisa.europa.eu/sites/default/files/2025-02/Finance%20TL%202024_Final.pdf.